

Lab 08 – Reinforcement Learning

1. Introduction

This lab focuses on reinforcement learning techniques using Markov Decision Processes (MDPs), Q-Learning, and Deep Q-Learning (DQN). The practical work was completed in Google Colab by completing the provided notebooks, running the algorithms, observing their behaviour, and comparing their performance.

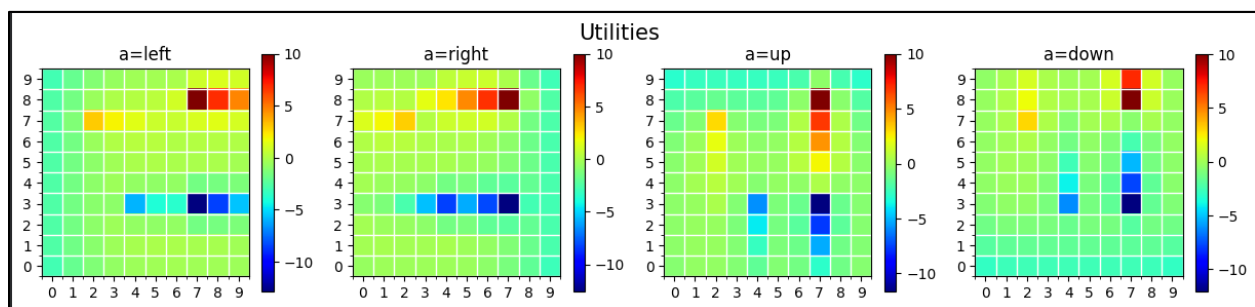
The main objectives were to understand policy evaluation, value iteration, policy iteration, tabular Q-Learning, the effect of increasing the GridWorld size, the difference between model-based and model-free reinforcement learning, and the effect of different epsilon values in a DQN model.

2. Question 1 – Markov Decision Process and Q-Learning

Question 1 was completed using the Markov Decision Process notebook and the GridWorld notebook. The missing code sections were completed and the notebooks were executed successfully.

- **2.1 Iterative Policy Evaluation**

Iterative policy evaluation was used to estimate the utility of each state for a fixed action/policy. The update repeatedly combines the immediate reward with the discounted expected utility of successor states. The results below show the utility distributions for the four actions: left, right, up, and down.



Observation: The utility maps differ for each action because each action produces different state transitions and future rewards. Positive reward areas obtain higher utility values, while states near negative rewards obtain lower values.

Observation: At 5 iterations, the utility map and action directions have become structured, showing that the policy has been repeatedly evaluated and improved.

2.4 Random Agent in GridWorld

The Random Agent selects one of the available actions without using previous experience. It was used as a baseline before evaluating the Q-Learning agent.

```

  Run Random Agent

  • Random agent moves randomly and does not learn from it's actions.
  • This gives a base performance to compare the Q-Learning agent to

  [6]
  ✓ Os
  ▶ env = GridWorld()
    agent = RandomAgent()

    print("Current position of the agent =", env.current_location)
    print(env.agent_on_map())
    available_actions = env.get_available_actions()
    print("Available_actions =", available_actions)
    chosen_action = agent.choose_action(available_actions)
    print("Randomly chosen action =", chosen_action)
    reward = env.make_step(chosen_action)
    print("Reward obtained =", reward)
    print("Current position of the agent =", env.current_location)
    print(env.agent_on_map())

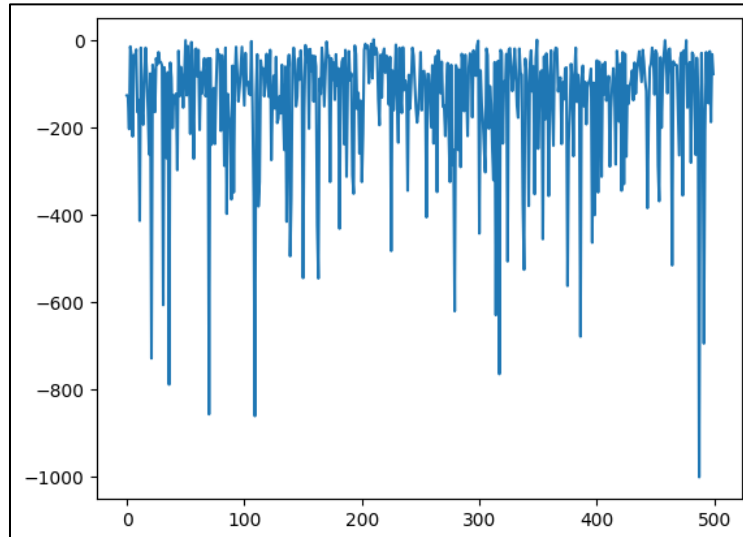
  ... Current position of the agent = (4, 2)
  [[0. 0. 0. 0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0. 0. 0. 0.]
   [0. 0. 1. 0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0. 0. 0. 0.]]
  Available_actions = ['UP', 'DOWN', 'LEFT', 'RIGHT']
  Randomly chosen action = LEFT
  Reward obtained = -1.0
  Current position of the agent = (4, 1)
  [[0. 0. 0. 0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0. 0. 0. 0.]
   [0. 1. 0. 0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0. 0. 0. 0.]
   [0. 0. 0. 0. 0. 0. 0. 0.]]

```

The example interaction shows the current position, available actions, a randomly selected action, the obtained reward, and the resulting new position.

2.5 Random Agent Performance

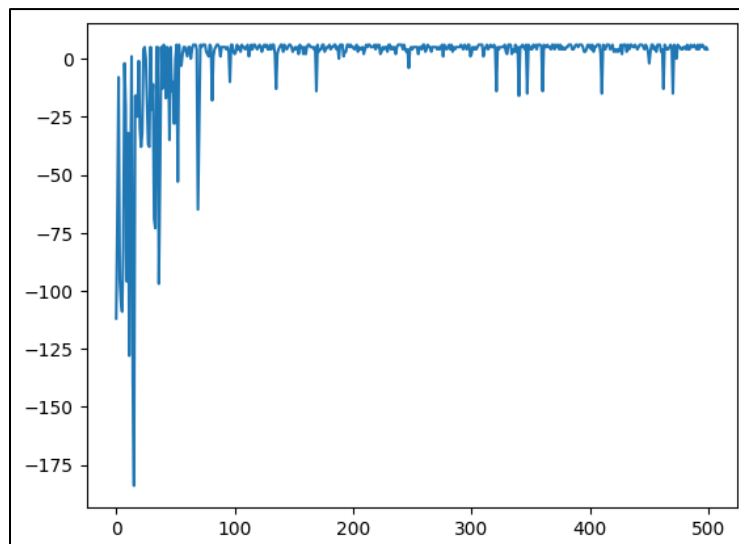
The Random Agent was executed for 500 trials. Because it does not learn from previous actions, the reward remains unstable and frequently drops to very low values.



Observation: The reward curve is highly inconsistent. This is expected because the agent continues to select random actions and does not improve its decision-making over time.

2.6 Q-Learning Agent Performance

The Q-Learning agent was then trained for 500 trials with learning enabled. Q-Learning updates the Q-value of each state-action pair using the current reward and the maximum estimated future Q-value. The update follows the standard form: $Q(s,a) = Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$.



Observation: The agent initially receives poor rewards while exploring the environment. As training continues, the reward improves and becomes more stable, indicating that the agent is learning a better policy. Compared with the Random Agent, the Q-Learning agent performs substantially better because it learns from previous interactions.

2.7 Larger GridWorld Experiment

To study the effect of environment size, the GridWorld was increased from 8×8 to 15×15 . The larger environment contains more states, which increases the amount of exploration and learning required.

```
Current position of the agent = (14, 0)
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
Available_actions = ['UP', 'DOWN', 'LEFT', 'RIGHT']
Randomly chosen action = LEFT
Reward obtained = -1.0
Current position of the agent = (14, 0)
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
```

Observation: Increasing the grid size increases the state space. Therefore, the agent generally requires more exploration and more training episodes to reach stable behaviour. Execution effort also increases because the agent must learn values for more state-action combinations.

3. Question 2 – Model-Based vs Model-Free Reinforcement Learning

A comparison was performed between Policy Iteration as a model-based method and Q-Learning as a model-free method. Execution time was measured for both approaches in the MDP environment.

```
[13] 2s
for episode in range(500):
    # Start from a random state
    s = np.random.choice(mdp.S)

    for step in range(100):
        # Epsilon-greedy action selection
        if np.random.rand() < epsilon:
            a = np.random.randint(len(mdp.A))
        else:
            a = np.argmax(Q[s])

        # Select one possible next state
        next_states = mdp.nextStates[(s, a)]

        if len(next_states) == 0:
            break

        s1 = np.random.choice(next_states)

        reward = mdp.R[s, a]

        # Q-learning update
        Q[s, a] = Q[s, a] + alpha * (
            reward
            + gamma * np.max(Q[s1])
            - Q[s, a]
        )

        s = s1

    model_free_time = time.time() - start_time

    # -----
    # DISPLAY RESULTS
    # -----

    print("Model-Based (Policy Iteration) Time:",
          round(model_based_time * 1000, 2), "ms")

    print("Model-Free (Q-Learning) Time:",
          round(model_free_time * 1000, 2), "ms")

... Model-Based (Policy Iteration) Time: 50.63 ms
   ... Model-Free (Q-Learning) Time: 675.96 ms
```

Approach	Method	Measured Time
Model-Based	Policy Iteration	50.63 ms
Model-Free	Q-Learning	675.96 ms

Model-Based Reinforcement Learning uses a known model of the environment, including transition behaviour and rewards. Policy Iteration and Value Iteration are examples of model-based methods. Because the model is available, planning can be performed directly.

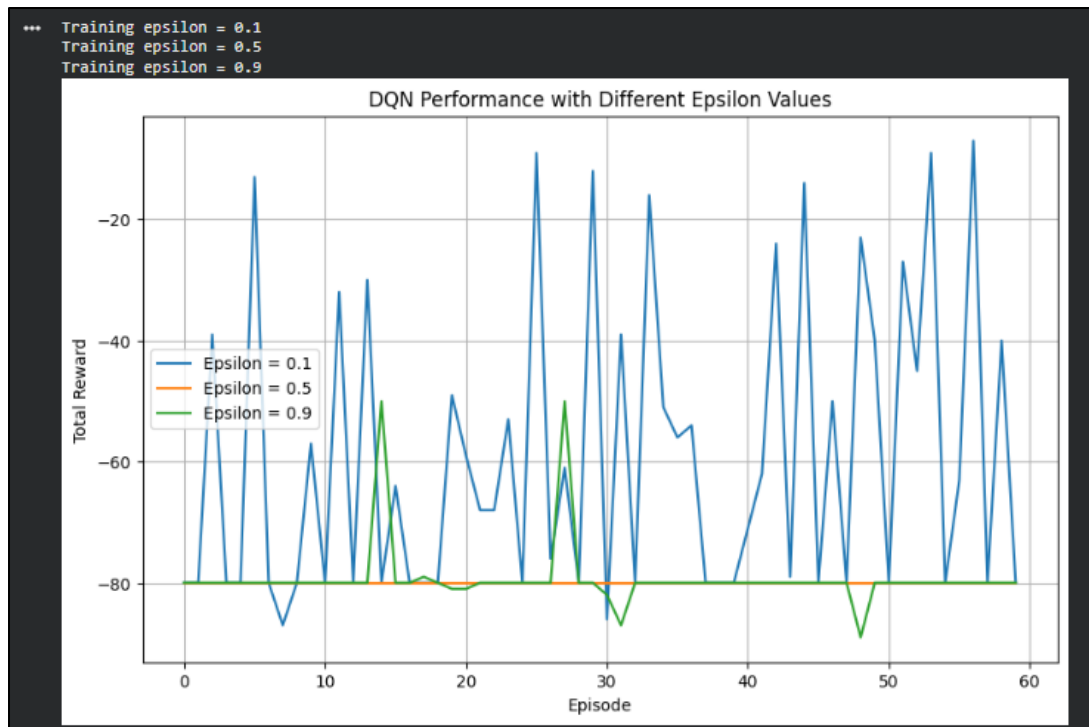
Model-Free Reinforcement Learning does not require a complete transition model. Instead, the agent learns through repeated interaction with the environment. Q-Learning is a model-free method because it updates Q-values from observed rewards and successor states.

Observation: In this run, Policy Iteration completed in 50.63 ms while Q-Learning took 675.96 ms. The model-free method required substantially more repeated updates because it learned from simulated experience instead of directly using the known model.

4. Question 3 – Deep Q-Learning (DQN)

A Deep Q-Learning model was implemented to replace the tabular Q-value representation with a neural network. The neural network receives the current GridWorld state as input and outputs estimated Q-values for the four actions: UP, DOWN, LEFT, and RIGHT.

An epsilon-greedy strategy was used to balance exploration and exploitation. Three epsilon values were evaluated: 0.1, 0.5, and 0.9.



Observation: In this experiment, epsilon = 0.1 produced the strongest reward performance. With a low epsilon, the agent mostly exploited learned actions while still performing limited exploration. Epsilon = 0.5 introduced more exploration, while epsilon = 0.9 caused very frequent exploration and therefore less opportunity to exploit the learned policy during the limited training run.

The result demonstrates the exploration–exploitation trade-off. Lower epsilon values favour exploitation, while higher values favour exploration. DQN differs from traditional Q-Learning because Q-values are approximated by a neural network rather than stored directly in a lookup table.

5. Conclusion

This lab demonstrated several core reinforcement learning approaches. Policy Evaluation, Value Iteration, and Policy Iteration showed how a known environment model can be used to calculate utilities and improve policies. The Random Agent provided a baseline with unstable performance, while Q-Learning improved over repeated interaction and converged toward a better policy.

The larger GridWorld experiment showed that increasing the number of states increases the exploration and learning effort. The comparison between Policy Iteration and Q-Learning highlighted the difference between model-based and model-free learning. Finally, the DQN experiment demonstrated how a neural network can approximate Q-values and how epsilon controls the balance between exploration and exploitation.